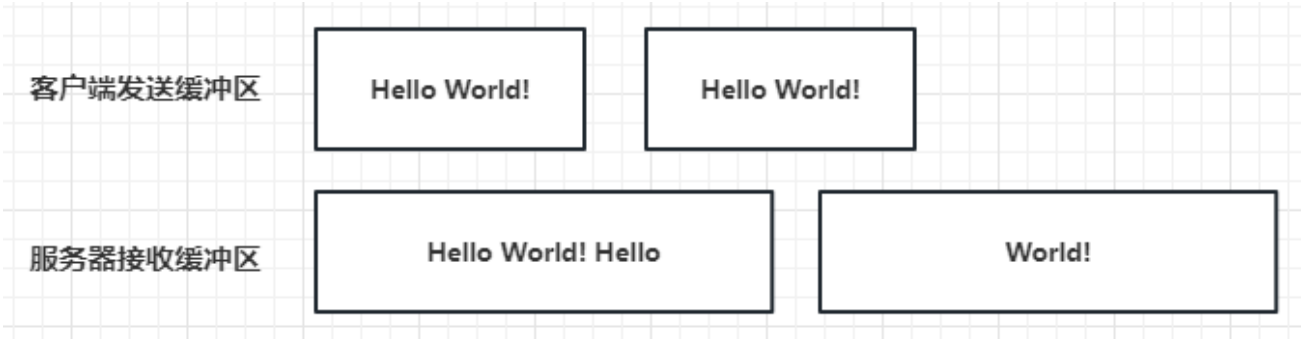
处理网络粘包问题

Posted on 2023-05-07 20:01:20 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2PSqYnkrogKeDPjv3gdBUAcbN5P)

粘包问题

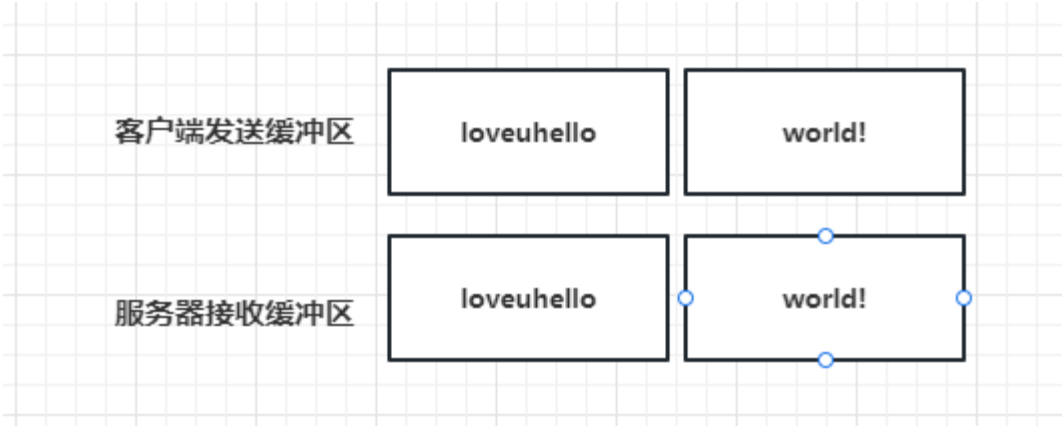
今天介绍一下如何处理粘包，粘包问题是服务器收发数据常遇到的一个现象，下面我们介绍一下粘包问题是什么，当客户端发送多个数据包给服务器时，服务器底层的tcp接收缓冲区收到的数据为粘连在一起的，如下图所示



当客户端发送两个Hello World！给服务器，服务器TCP接收缓冲区接收了两次，一次是Hello World!Hello, 第二次是World！。

粘包原因

因为TCP底层通信是面向字节流的，TCP只保证发送数据的准确性和顺序性，字节流以字节为单位，客户端每次发送N个字节给服务端，N取决于当前客户端的发送缓冲区是否有数据，比如发送缓冲区总大小为10个字节，当前有5个字节数据(上次要发送的数据比如'loveu')未发送完，那么此时只有5个字节空闲空间，我们调用发送接口发送hello world！其实就是只能发送Hello给服务器，那么服务器一次性读取到的数据就很可能是loveuhello。而剩余的world！只能留给下一次发送，下一次服务器接收到的就是world！
如下图



这是最好理解的粘包问题的产生原因。还有一些其他的原因比如

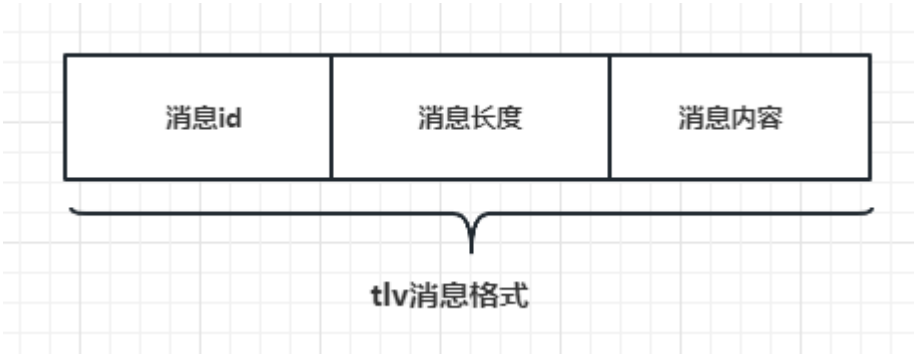
1　客户端的发送频率远高于服务器的接收频率，就会导致数据在服务器的tcp接收缓冲区滞留形成粘连，比如客户端1s内连续发送了两个hello world！,服务器过了2s才接收数据，那一次性读出两个hello world！。

2　tcp底层的安全和效率机制不允许字节数特别少的小包发送频率过高，tcp会在底层累计数据长度到一定大小才一起发送，比如连续发送1字节的数据要累计到多个字节才发送，可以了解下tcp底层的Nagle算法。

3　再就是我们提到的最简单的情况，发送端缓冲区有上次未发送完的数据或者接收端的缓冲区里有未取出的数据导致数据粘连。

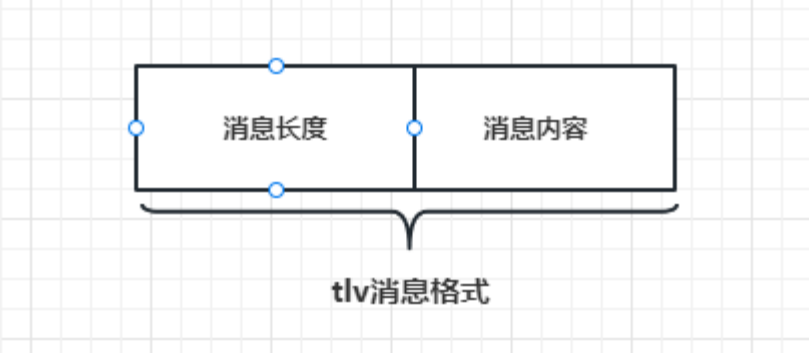
处理粘包

处理粘包的方式主要采用应用层定义收发包格式的方式，这个过程俗称切包处理，常用的协议被称为tlv协议(消息id+消息长度+消息内容)，如下图



为保证大家容易理解，我们先简化发送的格式，格式变为消息长度+消息内容的方式，之后再完善为tlv格式。

简化后的结构如下图



完善消息节点

之前我们设计过消息节点的数据结构MsgNode，这里需要完善一下.

```
class MsgNode
{
    friend class CSession;
public:
    MsgNode(char * msg, short max_len):_total_len(max_len + HEAD_LENGTH),_cur_len(0){
        _data = new char[_total_len+1]();
        memcpy(_data, &max_len, HEAD_LENGTH);
        memcpy(_data+ HEAD_LENGTH, msg, max_len);
        _data[_total_len] = '\0';
    }

    MsgNode(short max_len):_total_len(max_len),_cur_len(0) {
        _data = new char[_total_len +1]();
    }

    ~MsgNode() {
        delete[] _data;
    }

    void Clear() {
        ::memset(_data, 0, _total_len);
        _cur_len = 0;
    }
private:
    short _cur_len;
    short _total_len;
    char* _data;
};
```

1 两个参数的构造函数做了完善，之前的构造函数通过消息首地址和长度构造节点数据，现在需要在构造节点的同时把长度信息也写入节点,该构造函数主要用来发送数据时构造发送信息的节点。

2 一个参数的构造函数为较上次新增的，主要根据消息的长度构造消息节点，该构造函数主要是接收对端数据时构造接收节点调用的。

3 新增一个Clear函数清除消息节点的数据，主要是避免多次构造节点造成开销。

CSession类完善

为能够对收到的数据切包处理，需要定义一个消息接收节点，一个bool类型的变量表示头部是否解析完成，以及将处理好的头部先缓存起来的结构。

```
1.  //收到的消息结构
2.  std::shared_ptr<MsgNode> _recv_msg_node;
3.  bool _b_head_parse;
4.  //收到的头部结构
5.  std::shared_ptr<MsgNode> _recv_head_node;
```

_recv_msg_node用来存储接受的消息体信息
_recv_head_node用来存储接收的头部信息
_b_head_parse表示是否处理完头部信息

同时我们新增一个HEAD_LENGTH变量表示数据包头部的大小，修改原消息最大长度为1024*2

```
1. #define MAX_LENGTH  1024*2
2. #define HEAD_LENGTH 2
```

完善接收逻辑

我们需要修改HandleRead函数

```
1. void CSession::HandleRead(const boost::system::error_code& error, size_t  bytes_transferred, std::shared_ptr<CSession> shared_self){
2.     if (!error) {
3.         //已经移动的字符数
4.         int copy_len = 0;
5.         while (bytes_transferred>0) {
6.             if (!_b_head_parse) {
7.                 //收到的数据不足头部大小
8.                 if (bytes_transferred + _recv_head_node->_cur_len < HEAD_LENGTH) {
9.                     memcpy(_recv_head_node->_data + _recv_head_node->_cur_len, _data+ copy_len, bytes_transferred);
10.                    _recv_head_node->_cur_len += bytes_transferred;
11.                    ::memset(_data, 0, MAX_LENGTH);
12.                    _socket.async_read_some(boost::asio::buffer(_data, MAX_LENGTH),
13.                        std::bind(&CSession::HandleRead, this, std::placeholders::_1, std::placeholders::_2, shared_self));
14.                    return;
15.                }
16.                //收到的数据比头部多
17.                //头部剩余未复制的长度
18.                int head_remain = HEAD_LENGTH - _recv_head_node->_cur_len;
19.                memcpy(_recv_head_node->_data + _recv_head_node->_cur_len, _data+copy_len, head_remain);
20.                //更新已处理的data长度和剩余未处理的长度
21.                copy_len += head_remain;
22.                bytes_transferred -= head_remain;
23.                //获取头部数据
24.                short data_len = 0;
25.                memcpy(&data_len, _recv_head_node->_data, HEAD_LENGTH);
26.                cout << "data_len is " << data_len << endl;
27.                //头部长度非法
28.                if (data_len > MAX_LENGTH) {
29.                    std::cout << "invalid data length is " << data_len << endl;
30.                    _server->ClearSession(_uuid);
31.                    return;
32.                }
33.                _recv_msg_node = make_shared<MsgNode>(data_len);
34.
35.                //消息的长度小于头部规定的长度，说明数据未收全，则先将部分消息放到接收节点里
36.                if (bytes_transferred < data_len) {
37.                    memcpy(_recv_msg_node->_data + _recv_msg_node->_cur_len, _data + copy_le
```

```
n, bytes_transferred);
38.                 _recv_msg_node->_cur_len += bytes_transferred;
39.                 ::memset(_data, 0, MAX_LENGTH);
40.                 _socket.async_read_some(boost::asio::buffer(_data, MAX_LENGTH),
41.                     std::bind(&CSession::HandleRead, this, std::placeholders::_1, std::placehold
ers::_2, shared_self));
42.                 //头部处理完成
43.                 _b_head_parse = true;
44.                 return;
45.             }
46.
47.             memcpy(_recv_msg_node->_data + _recv_msg_node->_cur_len, _data + copy_len, d
ata_len);
48.             _recv_msg_node->_cur_len += data_len;
49.             copy_len += data_len;
50.             bytes_transferred -= data_len;
51.             _recv_msg_node->_data[_recv_msg_node->_total_len] = '\0';
52.             cout << "receive data is " << _recv_msg_node->_data << endl;
53.             //此处可以调用Send发送测试
54.             Send(_recv_msg_node->_data, _recv_msg_node->_total_len);
55.             //继续轮询剩余未处理数据
56.             _b_head_parse = false;
57.             _recv_head_node->Clear();
58.             if (bytes_transferred <= 0) {
59.                 ::memset(_data, 0, MAX_LENGTH);
60.                 _socket.async_read_some(boost::asio::buffer(_data, MAX_LENGTH),
61.                     std::bind(&CSession::HandleRead, this, std::placeholders::_1, std::placehold
ers::_2, shared_self));
62.                 return;
63.             }
64.             continue;
65.         }
66.
67.         //已经处理完头部，处理上次未接受完的消息数据
68.         //接收的数据仍不足剩余未处理的
69.         int remain_msg = _recv_msg_node->_total_len - _recv_msg_node->_cur_len;
70.         if (bytes_transferred < remain_msg) {
71.             memcpy(_recv_msg_node->_data + _recv_msg_node->_cur_len, _data + copy_len, b
ytes_transferred);
72.             _recv_msg_node->_cur_len += bytes_transferred;
73.             ::memset(_data, 0, MAX_LENGTH);
74.             _socket.async_read_some(boost::asio::buffer(_data, MAX_LENGTH),
```

```
                std::bind(&CSession::HandleRead, this, std::placeholders::_1, std::placeholders::_2, shared_self));
                return;
            }
            memcpy(_recv_msg_node->_data + _recv_msg_node->_cur_len, _data + copy_len, remain_msg);
            _recv_msg_node->_cur_len += remain_msg;
            bytes_transferred -= remain_msg;
            copy_len += remain_msg;
            _recv_msg_node->_data[_recv_msg_node->_total_len] = '\0';
            cout << "receive data is " << _recv_msg_node->_data << endl;
            //此处可以调用Send发送测试
            Send(_recv_msg_node->_data, _recv_msg_node->_total_len);
            //继续轮询剩余未处理数据
            _b_head_parse = false;
            _recv_head_node->Clear();
            if (bytes_transferred <= 0) {
                ::memset(_data, 0, MAX_LENGTH);
                _socket.async_read_some(boost::asio::buffer(_data, MAX_LENGTH),
                    std::bind(&CSession::HandleRead, this, std::placeholders::_1, std::placeholders::_2, shared_self));
                return;
            }
            continue;
        }
    }
    else {
        std::cout << "handle read failed, error is " << error.what() << endl;
        Close();
        _server->ClearSession(_uuid);
    }
}
```

1 copy_len记录的是已经处理过数据的长度，因为存在一次接收多个包的情况，所以copy_len用来做已经处理的数据长度的。

2 首先判断_b_head_parse是否为false，如果为false则说明头部未处理，先判断接收的数据是否小于头部, 如果小于头部大小则将接收到的数据放入_recv_head_node节点保存，然后继续调用读取函数监听对端发送数据。否则进入步骤3.

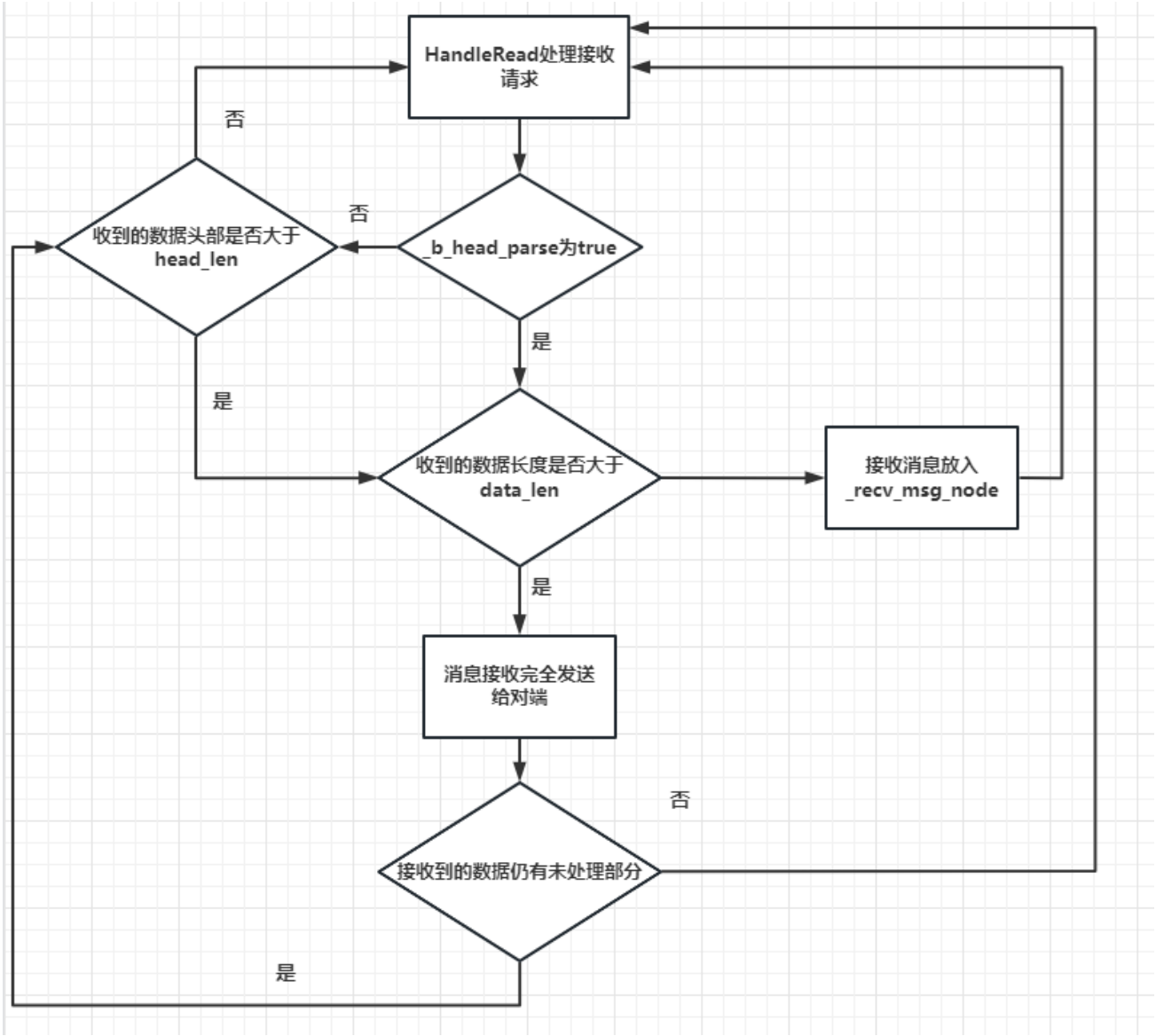
3 如果收到的数据比头部多，可能是多个逻辑包，所以要做切包处理。根据之前保留在_recv_head_node的长度，计算出剩余未取出的头部长度，然后取出剩余的头部长度保存在_recv_head_node节点，然后通过memcpy方式从节点拷贝出数据写入short类型的data_len

里，进而获取消息的长度。接下来继续处理包体，也就是消息体，判断接收到的数据未处理部分的长度和总共要接收的数据长度大小，如果小于总共要接受的长度，说明消息体没接收完，则将未处理部分先写入_recv_msg_node里，并且继续监听读事件。否则说明消息体接收完全，进入步骤4

4 将消息体数据接收到_recv_msg_node中，接受完全后返回给对端。当然存在多个逻辑包粘连，此时要判断bytes_transferred是否小于等于0，如果是说明只有一个逻辑包，我们处理完了，继续监听读事件，就直接返回即可。否则说明有多个数据包粘连，就继续执行上述操作。

5 因为存在_b_head_parse为true，也就是包头接收并处理完的情况，但是包体未接受完，再次触发HandleRead，此时要继续处理上次未接受完的消息体，大体逻辑和3，4一样。

以上就是处理粘包的过程，我们绘制流程图更明了一些



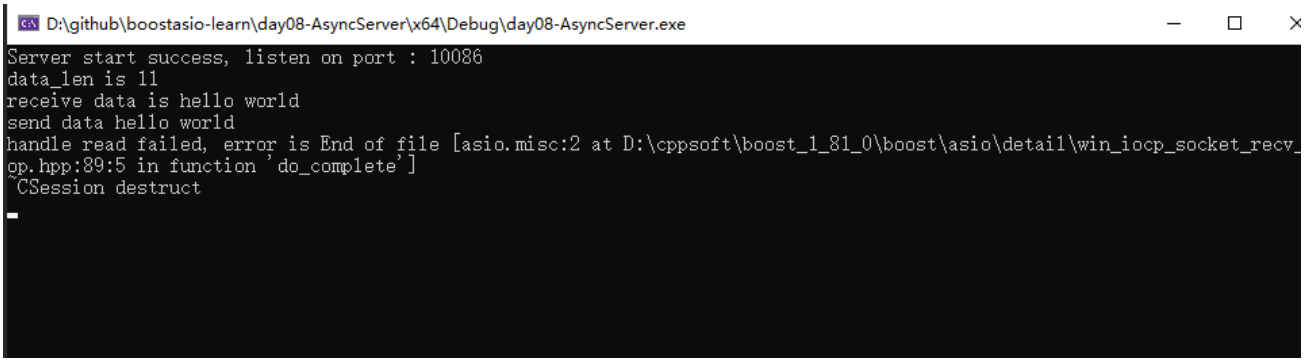
客户端修改

客户端的发送也要遵循先发送数据2个字节的数据长度，再发送数据消息的结构。
接收时也是先接收两个字节数据获取数据长度，再根据长度接收消息。

```
1. int main()
2. {
3.     try {
4.         //创建上下文服务
5.         boost::asio::io_context   ioc;
6.         //构造endpoint
7.         tcp::endpoint  remote_ep(address::from_string("127.0.0.1"), 10086);
8.         tcp::socket  sock(ioc);
9.         boost::system::error_code   error = boost::asio::error::host_not_found; ;
10.        sock.connect(remote_ep, error);
11.        if (error) {
12.            cout << "connect failed, code is " << error.value() << " error msg is " << error.message
    ();
13.            return 0;
14.        }
15.
16.        std::cout << "Enter message: ";
17.        char request[MAX_LENGTH];
18.        std::cin.getline(request, MAX_LENGTH);
19.        size_t request_length = strlen(request);
20.        char send_data[MAX_LENGTH] = { 0 };
21.        memcpy(send_data, &request_length, 2);
22.        memcpy(send_data + 2, request, request_length);
23.        boost::asio::write(sock, boost::asio::buffer(send_data, request_length+2));
24.
25.        char reply_head[HEAD_LENGTH];
26.        size_t reply_length = boost::asio::read(sock,boost::asio::buffer(reply_head, HEAD_LENG
    TH));
27.        short msglen = 0;
28.        memcpy(&msglen, reply_head, HEAD_LENGTH);
29.        char msg[MAX_LENGTH] = { 0 };
30.        size_t  msg_length = boost::asio::read(sock,boost::asio::buffer(msg, msglen));
31.
32.        std::cout << "Reply is: ";
33.        std::cout.write(msg, msglen) << endl;
34.        std::cout << "Reply len is " << msglen;
35.        std::cout << "\n";
36.    }
37.    catch (std::exception& e) {
38.        std::cerr << "Exception: " << e.what() << endl;
39.    }
40.    return 0;
```

```
41. }
```

服务器启动后，启动客户端，然后客户端发送Hello World，服务器收到后打印如下



粘包测试

为了测试粘包，需要制造粘包产生的现象，可以让客户端发送的频率高一些，服务器接收的频率低一些，这样造成前后端收发数据不一致导致多个数据包在服务器tcp缓冲区滞留产生粘包现象。
测试粘包之前，在服务器的CSession类里添加打印二进制数据的函数，便于查看缓冲区的数据

```
1. void CSession::PrintRecvData(char* data, int length) {
2.     stringstream ss;
3.     string result = "0x";
4.     for (int i = 0; i < length; i++) {
5.         string hexstr;
6.         ss << hex << std::setw(2) << std::setfill('0') << int(data[i]) << endl;
7.         ss >> hexstr;
8.         result += hexstr;
9.     }
10.    std::cout << "receive raw data is : " << result << endl;;
11. }
```

然后将这个函数放到HandleRead里，每次收到数据就调用这个函数打印接收到的最原始的数据，然后睡眠2秒再进行收发操作，用来延迟接收对端数据制造粘包，之后的逻辑不变

```
1. void CSession::HandleRead(const boost::system::error_code& error, size_t  bytes_transferred, std::shared_ptr<CSession> shared_self){
2.     if (!error) {
3.         PrintRecvData(_data, bytes_transferred);
4.         std::chrono::milliseconds dura(2000);
5.         std::this_thread::sleep_for(dura);
6.     }
7. }
```

修改客户端逻辑，实现收发分离。

```
1. int main()
2. {
3.     try {
4.         //创建上下文服务
5.         boost::asio::io_context    ioc;
6.         //构造endpoint
7.         tcp::endpoint  remote_ep(address::from_string("127.0.0.1"), 10086);
8.         tcp::socket  sock(ioc);
9.         boost::system::error_code    error = boost::asio::error::host_not_found; ;
10.        sock.connect(remote_ep, error);
11.        if (error) {
12.            cout << "connect failed, code is " << error.value() << " error msg is " << error.message
    ();
13.            return 0;
14.        }
15.
16.        thread send_thread([&sock] {
17.            for (;;) {
18.                this_thread::sleep_for(std::chrono::milliseconds(2));
19.                const char* request = "hello world!";
20.                size_t request_length = strlen(request);
21.                char send_data[MAX_LENGTH] = { 0 };
22.                memcpy(send_data, &request_length, 2);
23.                memcpy(send_data + 2, request, request_length);
24.                boost::asio::write(sock, boost::asio::buffer(send_data, request_length + 2));
25.            }
26.            });
27.
28.        thread recv_thread([&sock] {
29.            for (;;) {
30.                this_thread::sleep_for(std::chrono::milliseconds(2));
31.                cout << "begin to receive..." << endl;
32.                char reply_head[HEAD_LENGTH];
33.                size_t reply_length = boost::asio::read(sock, boost::asio::buffer(reply_head, HEAD
    _LENGTH));
34.                short msglen = 0;
35.                memcpy(&msglen, reply_head, HEAD_LENGTH);
36.                char msg[MAX_LENGTH] = { 0 };
37.                size_t  msg_length = boost::asio::read(sock, boost::asio::buffer(msg, msglen));
38.
39.                std::cout << "Reply is: ";
40.                std::cout.write(msg, msglen) << endl;
```

```
            std::cout << "Reply len is " << msglen;
            std::cout << "\n";
        }
        });

    send_thread.join();
    recv_thread.join();
}
catch (std::exception& e) {
    std::cerr << "Exception: " << e.what() << endl;
}
return 0;
}
```

再次启动服务器和客户端，看到粘包现象了，我们的服务器也能稳定切割数据包并返回正确的消息给客户端。
可以看到服务器收到了大量数据，然后准确切割返回给了客户端。如下图。



总结

该服务虽然实现了粘包处理，但是服务器仍存在不足，比如当客户端和服务器处于不同平台时收发数据会出现异常，根本原因是未处理大小端模式的问题，这个留给下节处理。
源码链接https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

热门评论

浏览(3623) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) 喜欢(532) (JavaScript:void(0);)

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

喜欢(587) 浏览(1595)

解密定时器的实现细节

喜欢(566) 浏览(1449)

Linux环境搭建和编码

喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



恋恋风辰

北京 朝阳



扫一扫上面的二维码图案，加我为朋友。

个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)

京ICP备2021033081号 (https://beian.miit.gov.cn/)